

SETUP GUIDE

Start to End — Employee Management System

Sri Annamalai Global Business Service

What's in This Guide

01 Before You Begin

02 Two Ways to Run the System

03 Path A — Docker Setup (Recommended)

04 Path B — Native Setup (Manual)

05 Creating the First Login

06 Adding Your Company's Data

07 Signing In and Checking It Works

08 The Business Rules, at a Glance

09 Backing Up the System

10 Troubleshooting

11 Where to Go Next

1. Before You Begin

What you need before you touch a terminal.

This guide walks through getting the Employee Management System running, start to end — from an empty computer to a working login with your company's own data in it.

You need three things before you start:

- The project folder — the whole employee-management-system folder, with the backend and frontend subfolders and docker-compose.yml inside it.
- A computer — Mac, Windows, or Linux. This guide uses Mac Terminal commands; on Windows, use PowerShell or Command Prompt instead — the commands themselves are the same.
- About 20–30 minutes the first time. After that, starting the system again takes seconds.

> Note: You do not need to know how to code to follow this guide. Every command is given to you exactly as you should type it.

2. Two Ways to Run the System

Pick one before you start — don't mix them.

There are two completely separate ways to get the system running. Both end up at the same place — a working app at http://localhost:5173 — but they get there differently.

	Path A: Docker (Recommended)	Path B: Native (Manual)
What you install	One program: Docker Desktop	Python, Node.js, PostgreSQL, and Redis — separately
Setup effort	Low — one command starts everything	Higher — each piece is installed and configured by hand
Best for	Most people, and any new computer (e.g. a client's)	Developers who want direct control, or are already set up this way
Starting it again later	One command: docker compose up -d	Two commands, in two terminal windows

Note: If you're not sure which to pick, use Path A (Docker). It's simpler, and it's the same way this system would be handed to a new client.

3. Path A — Docker Setup

Skip to Section 4 if you're using Path B instead.

Step 1: Install Docker Desktop (one-time only)

1. Go to docker.com/products/docker-desktop and download it for your computer.
2. Open the downloaded file and drag the Docker icon into your Applications folder (Mac) or run the installer (Windows).
3. Open Docker Desktop. It may ask for your password once, to install system-level helpers — that's normal.
4. Wait for the whale icon in your menu bar (Mac) or system tray (Windows) to stop animating. Docker is now ready.

Step 2: Start the system

1. Open Terminal (Mac) or PowerShell (Windows).
2. Go to the project folder.
3. Start everything.

```
cd "/path/to/employee-management-system"
docker compose up -d
```

The first time, this takes a few minutes — it's downloading and building the whole system. After that, starting it again takes only seconds.

Step 3: Confirm it's running

```
docker compose ps
```

You should see four services — postgres, redis, backend, and frontend — each showing Up (postgres and redis should also say healthy).

Step 4: Open it

Go to http://localhost:5173 in your browser. You'll see the sign-in page — but nobody can sign in yet. That's expected: continue to Section 5 to create the first login.

Note: Want the system on different port numbers (e.g. because something else is already using 5173 or 8000)? Edit the .env file at the very top of the project folder — change a number, save, and run docker compose up -d again.

4. Path B — Native Setup

Skip this whole section if you used Path A.

Step 1: Install the four pieces

- PostgreSQL (the database) — version 16 or newer
- Redis (caching) — version 7 or newer
- Python — version 3.12 or newer
- Node.js — version 20 or newer

On a Mac with Homebrew: `brew install postgresql@16 redis python node`

Step 2: Create the database

```
createdb employee_management_live
```

Step 3: Set up the backend

1. Go into the backend folder.
2. Create and activate a virtual environment — an isolated space for this project's Python packages.
3. Install the required packages.
4. Copy the example settings file and fill in your real database details.
5. Bring the database up to date (create all its tables).
6. Start the backend server.

```
cd backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
# edit .env: set DATABASE_URL, DATABASE_URL_SYNC, JWT_SECRET_KEY
alembic upgrade head
uvicorn app.main:app --reload
```

Important: Leave this terminal window open — closing it stops the backend. Open a second terminal window for the next step.

Step 4: Set up the frontend

1. In a new terminal window, go into the frontend folder.
2. Install the required packages.
3. Copy the example settings file.
4. Start the frontend server.

```
cd frontend
npm install
cp .env.example .env
npm run dev
```

Open http://localhost:5173 — you'll see the sign-in page. Continue to Section 5.

5. Creating the First Login

The one login that creates everyone else.

Right now, the system has a database with no one in it — not even an Admin. A dedicated, one-time script creates the very first login. It is interactive: it asks you to type an email and choose a password, and the password is never shown on screen, stored anywhere else, or visible to anyone but you.

Run it

If you used Path A (Docker):

```
docker compose exec -it backend python -m app.setup_live
```

If you used Path B (Native):

```
cd backend
source venv/bin/activate
python -m app.setup_live
```

What it asks

- An Admin email address — this is the login for whoever will administer the system.
- A password, typed twice (hidden as you type, as a security measure).

> Note: The password must be at least 12 characters, must not be a very common password, and must not contain the email's own name. This is deliberately stricter than everyone else's password (8 characters is enough for a normal employee login) — because this is the account that can create every other login.

It also sets up the basics every installation needs: the 7 roles, the 29 underlying permissions, and starter leave and document types. It creates no demo or fake data, and it refuses to run a second time if the database already has any login in it — so it can never be run by accident on a system already in use.

6. Adding Your Company's Data

Two ways in — pick whichever suits how much data you have.

Option 1: Enter it by hand, through the app

Sign in as the Admin you just created, and in this order:

1. Departments — the teams in your company (e.g. Data Entry, Transportation, HR).
2. Job Titles — the designations inside each department (e.g. "Data Entry Executive").
3. Employees — add each person, choosing their Department, Job Title, and Login Role.
4. Holidays — add the coming year's holidays; the attendance calendar is empty until you do.

This is the right choice for a small company, or for adding one or two new hires later.

Option 2: Bring in a whole list at once, from Excel

For a larger company starting fresh, an Excel file can load employees, their managers, roles, salaries, and any leave already taken this year, all in one step.

```
cd backend
source venv/bin/activate
python -m app.import_employees path/to/your_file.xlsx
```

Note: Departments and Job Titles should already exist (Option 1's first two steps) before you import — the import links each employee to them by name.

7. Signing In and Checking It Works

A short, real test — not just "does it load."

1. Open http://localhost:5173 and sign in with the Admin email and password from Section 5.
2. You should land on the Dashboard — empty numbers are expected on a brand-new system.
3. Add one Department (e.g. "Testing").
4. Add one Job Title under it.
5. Add one Employee, and give them a Login Role.
6. Open that employee's profile and confirm their details are there.

If all six steps work, the whole system — frontend, backend, and database — is genuinely working end to end, not just "the page loaded."

8. The Business Rules, at a Glance

What's built in, and where it's configured.

Rule	Value
Office start	9:00 AM, with a 10-minute grace period
Lunch	1:00–1:30 PM, deducted automatically from worked hours
Half day	Checking in after 1:30 PM, or working under 7h45 net
Standard shift	8h30 net of lunch — anything beyond is overtime
Weekly day off	Sunday
A paid month	Locked — no further changes once payroll is run and paid

These live in backend/.env (and its siblings .env.live, .env.demo, .env.example). Each one needs the same value — they are not shared automatically, so changing only one silently has no effect on the others.

See also: The full list of who can do what, role by role, is its own document: docs/01-roles-and-features.pdf, alongside this one.

9. Backing Up the System

Two scripts — one for each path from Section 2.

Both scripts save the database and every uploaded photo/document, write a plain-English summary of exactly what's inside (row counts, sizes, what changed since last time), and automatically keep only the last 8 backups so the folder doesn't grow forever.

Where backups are stored

Both land in a folder next to the project folder — not inside it — so a backup is never accidentally wiped out by re-cloning or rebuilding the project itself:

```
Desktop/Panneer Anna/
  employee-management-system/   <- the project
  WEEKLY_BACKUPS/                <- Path B backups land here
  DOCKER_BACKUPS/                <- Path A backups land here
```

Each backup is its own timestamped folder (e.g. 2026-02-14_110000/), containing:

- database.dump — the full database, in a format pg_restore can load back in
- uploads.tar.gz — every uploaded photo and document, archived together
- SUMMARY.txt — the plain-English summary (row counts, sizes, what changed since last time)
- manifest.json — the same summary, in a machine-readable form
- CHECKSUMS.txt — a fingerprint of the two files above, to confirm nothing got corrupted

If you used Path B (Native)

```
cd backend
source venv/bin/activate
python scripts/weekly_backup.py
```

Saves into WEEKLY_BACKUPS/, keeping the last 8.

If you used Path A (Docker)

The containers must be running first (`docker compose up -d`) — the script reaches the Docker database through them, the same way the app does.

```
cd backend
source venv/bin/activate
python scripts/docker_backup.py
```

Saves into DOCKER_BACKUPS/, also keeping the last 8 — kept completely separate from the native backups above, since it's a different database.

> Note: Either script can be run any time, as often as you like — both are read-only and completely safe. To run one automatically on a schedule (e.g. every Friday), add it to cron (Mac/Linux) or Task Scheduler (Windows).

10. Troubleshooting

Real problems, and the real fix — not guesses.

"service backend is not running" (Docker)

The containers have stopped — usually because the terminal window they were started in was closed. Run `docker compose up -d` again; it starts them in the background so they keep running even if you close the window.

"docker: command not found"

Docker Desktop is installed, but its command-line tool isn't on your terminal's PATH. It lives at `~/.docker/bin/docker` — use that full path, or add that folder to your PATH.

A port is already in use (e.g. 5432, 8000, or 5173)

Something else on the computer — often a native PostgreSQL install — is already using that port. Edit the .env file at the project's top level and change the matching port number (e.g. `POSTGRES_PORT`), then run `docker compose up -d` again.

Setup refuses the password: "That password is too weak"

The first Admin password must be at least 12 characters, not a common password, and must not contain the email's own name. Choose a longer, more random password and try again.

A scheduled backup never seems to run (Mac)

macOS sometimes blocks cron from actually running scheduled jobs until Terminal is given Full Disk Access in System Settings → Privacy & Security. The job can look correctly scheduled and still silently never fire until this is granted.

11. Where to Go Next

- docs/01-roles-and-features.pdf — exactly what each of the 7 Login Roles can see and do.
- Admin → Roles & Permissions, inside the app — a live, always-current, editable view of every permission, for whoever is signed in as Admin.
- This guide — come back to it any time the system needs to be set up again, on another computer, for another installation.

Sri Annamalai Global Business Service — Employee Management System